stabilizer-python: narrow structure report

This note summarizes the **layout** of `stabilizer-python/` and focuses on **phase / sign tracking** in the stabilizer tableau. It is intentionally short.

Package layout

Path	Role
`stabilizer_python/tableau.py`	Core simulator: **Aaronson–Gottesman-style tableau** (`StabilizerState`), gates, **Z measurement**, row ops, **phase bits**.
`stabilizer_python/circuit.py`	**Builder**: `Circuit` with `h`, `s`, `x`, `z`, `cnot`, `mz`; `run(state)` applies ops to `StabilizerState`.
`stabilizer_python/linear_algebra.py`	**GF(2)** helpers: `gaussian_elimination_gf2`, `rank_gf2` (used by tests / tooling, not inside every gate step).
`stabilizer_python/codes.py`	**Demos**: 3-qubit bit-flip code (`BitFlip3Code`), 9-qubit Shor encoder (`Shor9Code`).
`stabilizer_python/examples/`	Runnable demos (Bell, bit-flip, Shor).
`tests/`	Unit tests for tableau behavior, codes, and linear algebra.

Public API (`__init__.py`): `StabilizerState`, `Circuit`, `gaussian_elimination_gf2`, `rank_gf2`, and the `codes` submodule.

Phase and sign tracking (the important part)

The state is stored as a **2n × n** binary tableau: per-row Pauli on each qubit via `(x[r][q], z[r][q])` bits (X/Z decomposition). Rows `0..n-1` are **destabilizers**, rows `n..2n-1` are **stabilizers**.

`r_phase`: only global ± on each row

`StabilizerState` keeps `r_phase[r] ∈ {0,1}` interpreted as an overall **(−1)** sign on that row's Pauli (see docstring in `tableau.py`). The implementation does **not** store full **i**-phases on rows for general Pauli tracking; instead it uses a **mod-4 i-exponent only inside row multiplication**, then folds to **±** (`exp_i == 2` → flip sign). That is enough for the Clifford + Pauli measurement algebra used here.

`_row_mult_phase` (Pauli product phases)

When two single-qubit Pauli factors multiply, **order matters** (e.g. (XZ = iY) vs (ZX = -iY)). The helper `_row_mult_phase(x1,z1,x2,z2)` returns an **exponent of (i)** (mod 4) for that pair. `_rowmult` sums:

- existing row signs as +2 per row if `r_phase` is set,
- plus `_row_mult_phase` over all qubits,

then XORs the X and Z bit vectors and sets the new row sign from `exp_i % 4`.

Gate-induced sign updates

- **`h(q)`**: swaps X/Z on column `q`; if the cell is **Y** (`x & z`), flips `r_phase[r]` (comment: Y picks up a sign under H in this bookkeeping).
- **`s(q)`**: phase gate on Z leg (`z ^= x`); again **Y** flips `r_phase[r]`.
- **`x(q)` / `z(q)`**: Pauli conjugation flips sign when the row has **Z** or **X** on `q` respectively (Y is both, so it flips when the relevant bit is 1).
- **`cnot(c,t)`**: binary update of X/Z plus a **conditional sign** from the Aaronson–Gottesman rule (`if xc & zt & (xt ^ zc ^ 1): r_phase[r] ^= 1`).

Measurement (`measure_z`)

- **Random** branch: finds a stabilizer row with X on `q`, uses `_rowmult` to clear other Xs, swaps rows, rewrites tableau so the measured generator is (Z_q) with `r_phase[n+q] = outcome`.
- **Deterministic** branch: outcome is assembled from **phases of stabilizer rows** paired with destabilizers that have X on `q` (`outcome ^= r_phase[n + r]` pattern).

So **phase bits are load-bearing** for deterministic Z outcomes, not only for aesthetics.

Ancilla hygiene

`reset_z(q)` measures Z and applies `x(q)` if the outcome was 1, so the qubit is returned to (|0\rangle) when used as a **measured ancilla** (used in the bit-flip syndrome helper in `codes.py`).

Results (tests)

From repo root:

```
python -m pytest -q stabilizer-python/tests
...........                                                              [100%]
11 passed in ~0.03s
```

(Exact timing varies by machine.)

Novel / distinguishing points (for this codebase)

1. **Explicit mod-4 phase bookkeeping in `_rowmult`**, then **projection to a single sign bit per row** — a minimal compromise between correctness for row operations and a small memory footprint.
2. **Clear split** between immutable-style **`Circuit`** **recording** and mutable **`StabilizerState`** simulation — easy to read and to test.
3. **Pure Python, no NumPy** for the simulator core; GF(2) helpers are separate and reusable.
4. **Small QEC surface**: bit-flip and Shor **encoder / syndrome** patterns sit beside the core, showing how `measure_z` + `reset_z` compose for stabilizer extraction.

Limits (honest scope)

- Tracks ± on tableau rows, not arbitrary **(i^k)** phases on every generator in full generality (sufficient for the gates and measurements implemented).
- **Z-basis measurements only** via `measure_z` (appropriate for stabilizer / CHP-style demos).

For file references, the phase logic is concentrated in `stabilizer_python/tableau.py` (`_row_mult_phase`, `_rowmult`, `h`, `s`, `x`, `z`, `cnot`, `measure_z`).